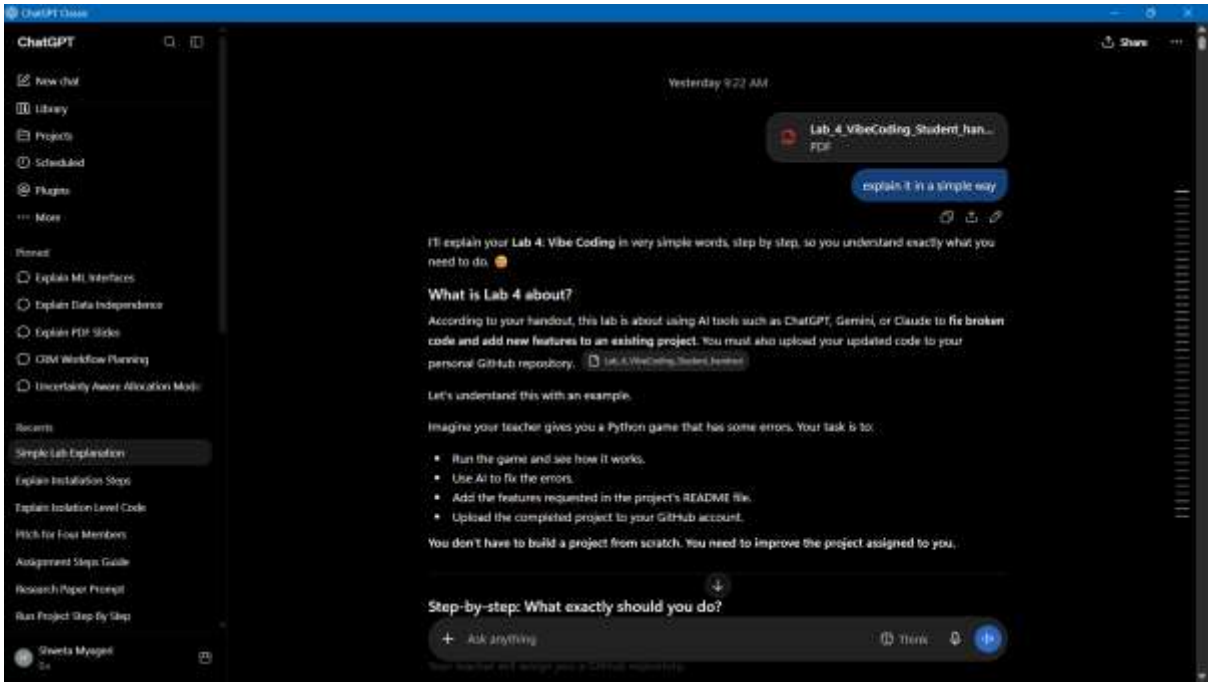


What is Lab 4 about?

According to your handout, this lab is about using AI tools such as ChatGPT, Gemini, or Claude to **fix broken code and add new features to an existing project**. You must also upload your updated code to your personal GitHub repository.

Let's understand this with an example.

Imagine your teacher gives you a Python game that has some errors. Your task is to:

- Run the game and see how it works.
- Use AI to fix the errors.
- Add the features requested in the project's README file.
- Upload the completed project to your GitHub account.

You don't have to build a project from scratch. You need to improve the project assigned to you.

Step-by-step: What exactly should you do?

Step 1: Open the GitHub repository assigned to you

Your teacher will assign you a GitHub repository.

A repository is simply a place where your project's code is stored.

For example, your teacher might give you a repository containing a Python puzzle game.

First, open the repository and look for a file called README.md.

Step 2: Read the README file

The README file explains what the project does and what you need to complete.

It tells you which features you must add.

For example, suppose your project is a puzzle game. Its README might ask you to:

- Fix a bug that prevents the game from starting.
- Add a score counter.
- Add a timer.
- Add a restart button.

Important: These are just examples. You must follow the actual requirements in your assigned repository's README file.

Step 3: Clone or fork the repository

Your handout gives you two options: clone or fork the repository.

Let's understand the difference.

Clone: Downloads a copy of the repository onto your computer.

Fork: Creates a copy of the repository under your own GitHub account.

For this lab, you need to work on your own copy and eventually push your changes to your personal repository.

If you are unsure which option your teacher expects, check the assigned repository's instructions.

Step 4: Run the Python project

After obtaining your copy, open the project in VS Code.

Find the instructions for running the code in the README file.

Run the game or puzzle.

Check whether it works correctly.

If it produces an error or a feature doesn't work, take note of the problem.

Don't start fixing anything yet! You need to record a video of the original project first.

Step 5: Record a 10-second video BEFORE making changes

Your handout specifically asks you to record a 10-second video before modifying the code.

This video shows the original condition of the project.

For example:

- Open the game.
- Show how it behaves.
- If it is broken, show the problem if possible.
- Record approximately 10 seconds.

Save the video somewhere you can find it later.

You will submit this video as evidence of the project's original state.

Step 6: Use AI to fix the broken code

Now comes the main part: **Vibe Coding**.

Vibe Coding means using natural-language instructions to tell an AI coding tool what changes you want.

Instead of manually figuring out every line of code, you explain the problem to the AI and ask it to help fix it.

For example, you could give ChatGPT this prompt:

Writing

Examine the Python code in my project and identify why the game is not working correctly.

First, explain the problem in simple words. Then suggest the smallest necessary code changes to fix it. Do not remove any existing features or modify unrelated files.

After making the changes, explain how I can test whether the fix works.

The AI can help you understand the problem and produce the corrected code.

You should then apply the changes, run the project again, and check whether the error has been fixed.

Important: Don't just copy the code and assume it works. Run it and verify the result.

Step 7: Add the features mentioned in the README

Once the broken code is fixed, add the required features.

Your handout says to write prompts to add the README features **one by one**.

Suppose your README requires three features:

Task	What you ask the AI to do
Feature 1	Add a score counter to the game.
Feature 2	Add a timer that tracks how long the player takes.
Feature 3	Add a restart button that resets the game.

These are example features, not necessarily the ones in your assigned project.

For each feature, follow this process:

1. Write a prompt explaining what you want.
2. Let the AI suggest or make the code changes.
3. Run the project.
4. Check whether the feature works.
5. Fix any remaining problems before moving to the next feature.

Why one feature at a time?

If you ask the AI to change everything at once, it may introduce errors or change existing functionality. Smaller changes make testing easier.

Step 8: Record another 10-second video AFTER making all the changes

Once you have fixed the bugs and added all the required features, run the updated project.

Record another 10-second video showing the improved version.

This is your **after video**.

You should now have two videos:

- **Before video:** The original project.
- **After video:** The updated project with the fixes and new features.

These videos help demonstrate the difference between the original and updated versions.

Step 9: Push your code to your personal GitHub repository

After completing the changes, upload your code to your own GitHub repository.

This process is called *pushing*.

If your project is already connected to your personal repository, you can generally use these commands in the VS Code terminal:

git add .

git commit -m "Fix broken game code"

git push

However, your handout requires **separate commits for each task**, so you should not put all your changes into a single commit.

For example:

git add .

git commit -m "Fix broken game code"

git push

Then, after completing Feature 1:

git add .

git commit -m "Add score counter"

git push

Then, after completing Feature 2:

git add .

git commit -m "Add game timer"

git push

Repeat this process for each remaining task.

These examples assume you have already configured Git and connected your local project to your personal repository.

Important: Each commit should represent a distinct task. Don't include changes for several unrelated tasks in one commit.

Step 10: Do NOT raise a pull request to the original repository

Your handout explicitly says not to raise a pull request to the main repository named SETAPESU26.

A pull request is a way of asking the owner of another repository to accept your changes.

For this assignment, you only need to push your work to your personal repository.

You do not need to ask the owner of the original repository to merge your changes.

What do you need to submit?

This is very important because completing the code is only part of the assignment.

Your handout requires three deliverables under the Lab-4 folder in your lab repository.

Deliverable	**What it means**
Before and after videos	Two short videos showing the original and improved project.
Updated code	The final version of your Python project.
Exported chat history	A document or PDF containing your conversations with the AI coding tool.

Your final folder could look something like this:

```
Your-Lab-Repository/
|
└── Lab-4/
    |
    ├── before.mp4
    ├── after.mp4
    ├── chat_history.pdf
    |
    └── project/
        ├── main.py
        ├── other_files.py
        └── README.md
```

This is only an example structure. Follow any additional folder requirements provided by your teacher.

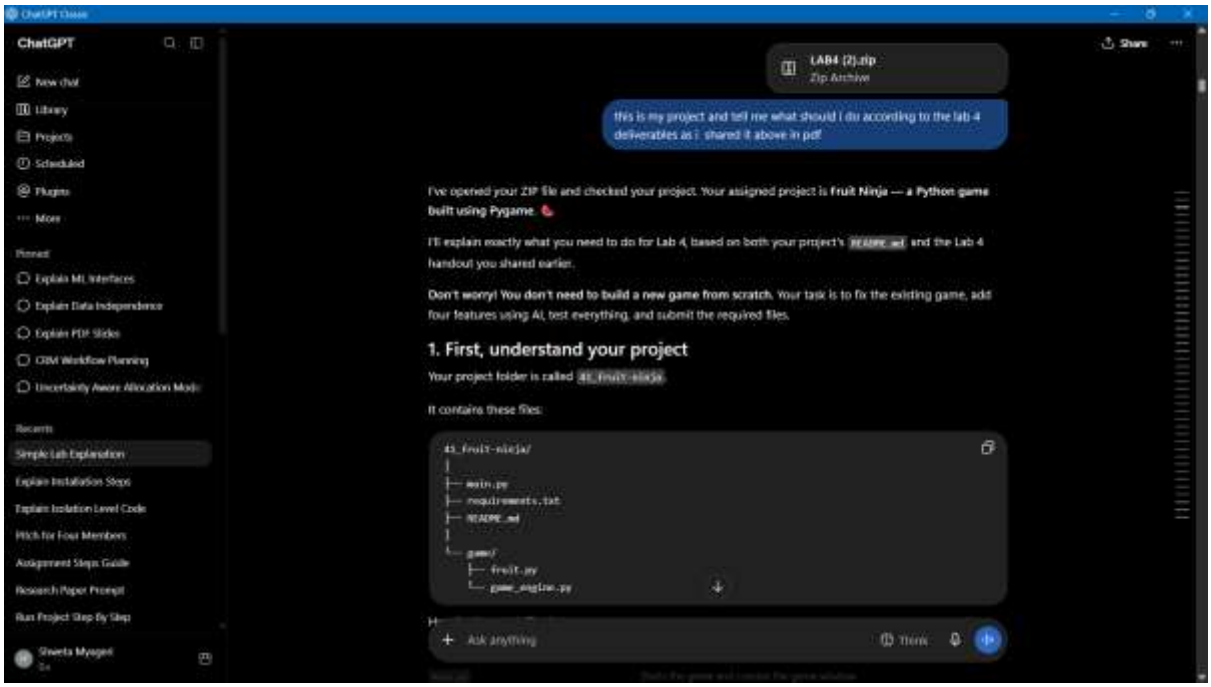
Make sure your exported chat history includes the prompts you used to fix the code and add the features.

What does your teacher expect you to demonstrate?

In simple words, your teacher wants to see that you can:

- Understand an existing Python project.
- Use AI tools to fix broken code.
- Write clear prompts that describe the required changes.
- Add the features specified in the README.
- Test the updated project.
- Use Git and GitHub to manage your work.
- Maintain separate commits for separate tasks.

The handout sets a duration of **60 minutes** and specifies that this is an **individual assignment**. You must complete the work yourself rather than working as a group.



I've opened your ZIP file and checked your project. Your assigned project is **Fruit Ninja — a Python game built using Pygame**. 🍉

I'll explain exactly what you need to do for Lab 4, based on both your project's README.md and the Lab 4 handout you shared earlier.

Don't worry! You don't need to build a new game from scratch. Your task is to fix the existing game, add four features using AI, test everything, and submit the required files.

1. First, understand your project

Your project folder is called 41_fruit-ninja.

It contains these files:

```
41_fruit-ninja/
|
├── main.py
├── requirements.txt
├── README.md
|
└── game/
    ├── fruit.py
    └── game_engine.py
```

Here is what each file does.

File	What it does
main.py	Starts the game and creates the game window.
fruit.py	Controls how fruits and bombs move and how they are detected when sliced.
game_engine.py	Controls the gameplay, score, lives, fruit spawning, and slicing.
requirements.txt	Lists the Python libraries required to run the game.
README.md	Explains the four tasks you must complete.

How does the game work?

Imagine a simple Fruit Ninja game.

Fruits and bombs appear at the bottom of the screen and move upwards.

You move your mouse across the screen like a knife.

- When you slice a fruit, your score increases.
- When you miss a fruit, you lose a life.
- When you slice a bomb, the game ends.
- You start with three lives.

Your task is to improve this existing game.

---

2. Your Lab 4 requires FOUR tasks

I checked your project's README.md. It specifically lists four tasks.

You must complete all four tasks, not just fix the existing code.

Task 1: Fix the fruit-slicing problem

What is the problem?

Currently, the game checks whether your mouse pointer is inside a fruit.

For example, imagine a fruit is at the centre of the screen.

You quickly move your mouse from left to right.

Your mouse might jump from one side of the fruit to the other between two frames.

Even though your mouse visually passes through the fruit, the game might not detect the slice.

What should you do?

Improve the collision detection so that the game checks the path of your mouse movement instead of checking only the mouse's current position.

This should allow fast mouse swipes to slice fruits more reliably.

Prompt to give ChatGPT

Use this prompt when you start working on Task 1.

Writing

I am working on a Python Pygame Fruit Ninja game for my Lab 4 Vibe Coding assignment.

Task 1: Refine Collision Detection.

Currently, the fruit is sliced only when a reported mouse position falls inside the fruit's circular area. Fast mouse swipes can pass through a fruit without slicing it.

Please inspect my existing fruit.py and game_engine.py files.

Fix the collision detection so that the game checks the line segment between the previous and current mouse positions against the fruit's circular area.

Requirements:

- Fast mouse swipes should reliably slice fruits.
- Do not change the existing scoring, lives, or fruit-spawning behaviour.
- Do not remove existing functionality.

- Keep the changes as small as possible.
- Explain which files you changed and why.
- Tell me how to test the fix.

Please provide the complete updated code for every file you modify.

After applying the changes: Run the game and try moving your mouse quickly across a fruit.

Check whether the fruit gets sliced even when your mouse moves quickly.

Once you have verified Task 1, create a separate Git commit for it.

Task 2: Add a proper Game Over screen

What is the problem?

Currently, when the game ends, your code prints something like this in the terminal:

Game over! Final score: 5

But the player should see the final score on the game screen.

What should you do?

When the player slices a bomb or loses all three lives, display a proper Game Over screen.

For example:

GAME OVER!

Final Score: 5

Thank you for playing!

The exact design is up to you.

The important thing is that the game displays the final score inside its window instead of relying only on a terminal message.

The game should also wait for the player to make a choice or close the window, rather than continuing normal gameplay.

Prompt to give ChatGPT

Writing

Task 2: Implement Game Over Condition in my existing Python Pygame Fruit Ninja game.

Currently, when the game ends, the final score is printed only in the terminal.

Modify the existing code to display a proper Game Over screen inside the Pygame window.

Requirements:

- The game ends immediately when the player slices a bomb.
- The game also ends when the player loses all lives.
- Display GAME OVER and the player's final score clearly on the screen.
- Stop spawning and updating fruits after the game ends.
- Allow the player to close the window normally.
- Prepare the code so that a replay option can be added in Task 3.
- Preserve the existing gameplay and collision-detection fix from Task 1.
- Explain the changes and provide the complete updated code for modified files.

After applying the changes: Play until you slice a bomb or lose all your lives.

Check whether the Game Over screen appears with your final score.

Create a separate Git commit for Task 2.

Task 3: Add a Replay option with difficulty selection

What is the problem?

Currently, once the game ends, there is no proper way to start a new game.

You would have to close and restart the application.

Your README requires a replay option.

What should you do?

After the Game Over screen appears, give the player three difficulty options:

Difficulty	Suggested behaviour
Easy	Fruits appear less frequently, with fewer bombs.
Medium	Normal fruit-spawning frequency and bomb chance.
Hard	Fruits appear more frequently, with a higher bomb chance.

These are suggested settings. The README requires the three difficulty levels but does not specify their exact values.

For example, your screen might display:

GAME OVER!

Final Score: 5

PLAY AGAIN

1. Easy
2. Medium
3. Hard

4. Exit

The player should be able to select a difficulty and start a new game.

When restarting, the game should reset the score and lives, clear the old fruits, and apply the selected difficulty.

Prompt to give ChatGPT

Writing

Task 3: Add a Replay Option with Difficulty Selection to my existing Python Pygame Fruit Ninja game.

After Game Over, display a menu with these options:

- Easy
- Medium
- Hard
- Exit

Requirements:

- Allow the player to select a difficulty using the keyboard.
- Easy should have a slower spawning rate and lower bomb chance.
- Medium should use the normal settings.
- Hard should have a faster spawning rate and higher bomb chance.

- Use clearly defined settings for each difficulty.
- When the player chooses a difficulty, restart the game without closing the application.
- Reset the score, lives, fruit list, mouse trail, and spawning timer.
- Ensure the new difficulty settings are applied correctly.
- Preserve the collision-detection fix from Task 1 and Game Over screen from Task 2.
- Keep the code organised and explain the changes.
- Provide the complete updated code for modified files.

After applying the changes: Test all three difficulty levels and confirm that replay works.

Create a separate Git commit for Task 3.

Task 4: Add sound effects

What is the problem?

Currently, the game does not have sound effects.

Your README requires sound feedback for three events.

What should you do?

Add sound effects for:

- **Fruit sliced:** Play a short sound when the player successfully slices a fruit.
- **Bomb hit:** Play a sound when the player slices a bomb.
- **Game Over:** Play a sound when the game ends.

You can use small sound files in WAV or another format supported by Pygame's mixer.

Keep the sound files inside your project folder so that the game can find them.

For example:

41_fruit-ninja/

│

├── main.py

├── requirements.txt

├── README.md

```
|
├── assets/
|   └── sounds/
|       ├── slice.wav
|       ├── bomb.wav
|       └── game_over.wav
|
└── game/
    ├── fruit.py
    └── game_engine.py
```

This is an example of how you could organise the sound files.

Prompt to give ChatGPT

Writing

Task 4: Add Sound Feedback to my existing Python Pygame Fruit Ninja game.

Requirements:

- Play a short sound when a fruit is successfully sliced.
- Play a different sound when a bomb is sliced.
- Play a sound when the Game Over screen appears.
- Avoid playing sounds repeatedly for the same event.
- Initialise the Pygame audio mixer safely.
- Store sound files in an organised assets/sounds folder.
- Handle missing sound files gracefully so that the game can still run.
- Update requirements.txt if any additional dependency is genuinely needed.
- Preserve all previous functionality, including collision detection, Game Over, replay, and difficulty selection.
- Explain where the sound files should be placed and how to test each sound.
- Provide the complete updated code for modified files.

After applying the changes: Play the game and test each of the three sound effects.

Create a separate Git commit for Task 4.

---

3. What should you do FIRST?

Before changing any code, follow this order.

Step A: Open the project in VS Code

Extract your ZIP file if you haven't already.

Open the 41_fruit-ninja folder in VS Code.

Open the terminal using:

Terminal → New Terminal

Make sure the terminal is in the project folder containing main.py.

Step B: Check Python

Run:

python --version

Your README requires Python 3.10 or later.

Step C: Install the dependencies

Run:

pip install -r requirements.txt

This installs Pygame, which the game uses.

If pip doesn't work, try:

python -m pip install -r requirements.txt

Step D: Run the original game

Run:

python main.py

Check whether the game opens and whether you can move the mouse to slice fruits.

If the game doesn't start, save the error message so we can investigate it.

Step E: Record the BEFORE video

Do this before fixing anything.

Record approximately 10 seconds of the original game.

Try to show the original gameplay and, if possible, the fast-swipe slicing problem.

Save it as:

before.mp4

If the game won't start, record the error and investigate it before proceeding. The aim is to capture the original state before making changes.

---

4. How to manage your Git commits

Your Lab 4 handout specifically requires a separate commit for every task.

You should aim for the following structure:

Commit	Work completed
Commit 1	Fix collision detection
Commit 2	Add Game Over screen
Commit 3	Add replay and difficulty selection
Commit 4	Add sound effects

If you need an additional commit to fix a problem, that's fine. The important thing is to keep the tasks separate.

For each completed task, you can use:

git status

git add game/fruit.py game/game_engine.py

git commit -m "Fix collision detection"

git push

The filenames in git add must match the files you actually changed.

For subsequent tasks, stage the appropriate changed files and use a descriptive commit message.

Important: Your uploaded ZIP contains an existing Git repository connected to the original SETAPESU26/41_fruit-ninja repository. Your handout says not to raise a pull request to the main repository.

Before pushing, we need to ensure that your local project is connected to **your personal GitHub repository**, not the original repository.

Do not push to the original repository.

---

5. What should your final submission contain?

Your Lab 4 handout requires three main deliverables:

1. Before and after videos

- before.mp4 — approximately 10 seconds showing the original game.
- after.mp4 — approximately 10 seconds showing the improved game and its new features.

2. Updated code

Your complete working project, including the Python files and any required sound assets.

3. Exported AI chat history

A document or PDF containing your prompts and the AI conversation used during the assignment.

Your project README also mentions submitting the AI chat page link with the complete chat history. Follow your instructor's exact submission instructions if both the link and an exported file are requested.

Put the deliverables inside the Lab-4 folder in your lab repository, as instructed in the handout.

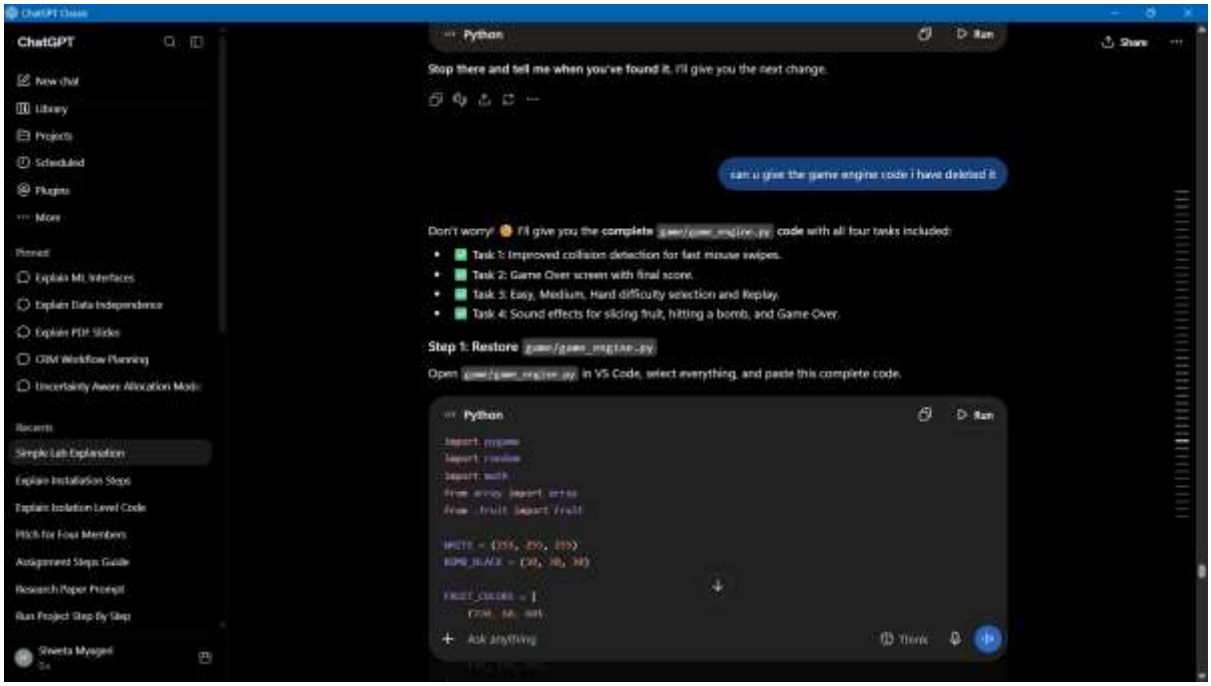
---

6. Your complete Lab 4 checklist

Use this checklist to track your progress.

Task	Status
Open the assigned project in VS Code	☐
Read the README	☑ Done — I've inspected it
Install dependencies	☐
Run the original game	☐
Record the 10-second BEFORE video	☐
Task 1: Fix collision detection	☐

Task	Status
Commit Task 1 separately	☐
Task 2: Add Game Over screen	☐
Commit Task 2 separately	☐
Task 3: Add replay and difficulty selection	☐
Commit Task 3 separately	☐
Task 4: Add sound effects	☐
Commit Task 4 separately	☐
Test all features	☐
Record the 10-second AFTER video	☐
Export the AI chat history	☐
Push code to your personal GitHub repository	☐
Upload all deliverables to the Lab-4 folder	☐



Don't worry! 😊 I'll give you the **complete game/game_engine.py code** with all four tasks included:

- ✅ Task 1: Improved collision detection for fast mouse swipes.
- ✅ Task 2: Game Over screen with final score.
- ✅ Task 3: Easy, Medium, Hard difficulty selection and Replay.
- ✅ Task 4: Sound effects for slicing fruit, hitting a bomb, and Game Over.

Step 1: Restore game/game_engine.py

Open game/game_engine.py in VS Code, select everything, and paste this complete code.

```
import pygame
import random
import math
from array import array
from .fruit import Fruit


WHITE = (255, 255, 255)
BOMB_BLACK = (30, 30, 30)


FRUIT_COLORS = [
    (220, 60, 60),
    (230, 140, 40),
    (230, 200, 40),
    (90, 180, 90),
]


DIFFICULTIES = {
    "Easy": {
        "spawn_interval": 70,
        "bomb_chance": 0.08,
        "speed_scale": 0.8,
    },
```

```
    "Medium": {
        "spawn_interval": 55,
        "bomb_chance": 0.15,
        "speed_scale": 1.0,
    },
    "Hard": {
        "spawn_interval": 35,
        "bomb_chance": 0.25,
        "speed_scale": 1.3,
    },
}



def segment_intersects_circle(start, end, center, radius):
    """Check whether a mouse movement segment crosses a fruit."""
    sx, sy = start
    ex, ey = end
    cx, cy = center

    dx = ex - sx
    dy = ey - sy

    if dx == 0 and dy == 0:
        return (sx - cx) ** 2 + (sy - cy) ** 2 <= radius ** 2

    t = ((cx - sx) * dx + (cy - sy) * dy) / (
        dx * dx + dy * dy
    )
```

```
    t = max(0.0, min(1.0, t))

    closest_x = sx + t * dx
    closest_y = sy + t * dy

    return (
        (closest_x - cx) ** 2 + (closest_y - cy) ** 2
        <= radius ** 2
    )


def create_tone(frequency, duration=0.15, volume=0.35):
    """Generate a simple sound without requiring external audio files."""
    sample_rate = 22050
    samples = int(sample_rate * duration)
    amplitude = int(32767 * volume)

    sound_data = array("h")

    for i in range(samples):
        time_value = i / sample_rate

        # Fade the sound out smoothly.
        fade = 1.0 - (i / samples)

        sample = int(
            amplitude
```

```
            * fade
            * math.sin(2 * math.pi * frequency * time_value)
        )

        sound_data.append(sample)

    return pygame.mixer.Sound(buffer=sound_data.tobytes())


class GameEngine:

    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.font = pygame.font.SysFont("Arial", 28)
        self.title_font = pygame.font.SysFont("Arial", 46)
        self.message_font = pygame.font.SysFont("Arial", 24)
        self.small_font = pygame.font.SysFont("Arial", 20)

        # Sound effects
        self.sound_enabled = False
        self.slice_sound = None
        self.bomb_sound = None
        self.game_over_sound = None

        try:
            if pygame.mixer.get_init() is None:
```

```
            pygame.mixer.init(
                frequency=22050,
                size=-16,
                channels=1,
            )

        self.slice_sound = create_tone(850, 0.10, 0.25)
        self.bomb_sound = create_tone(110, 0.35, 0.45)
        self.game_over_sound = create_tone(260, 0.45, 0.35)

        self.sound_enabled = True

    except pygame.error:
        print("Sound effects are unavailable. The game will continue silently.")

    self.selected_difficulty = "Medium"
    self.state = "menu"

    self.fruits = []
    self.trail = []

    self.score = 0
    self.lives = 3
    self.game_over_reason = None

    self.apply_difficulty()

def play_sound(self, sound):
```

```
    """Play a sound safely if audio is available."""
    if self.sound_enabled and sound is not None:
        try:
            sound.play()
        except pygame.error:
            pass

def apply_difficulty(self):
    settings = DIFFICULTIES[self.selected_difficulty]

    self.spawn_interval = settings["spawn_interval"]
    self.bomb_chance = settings["bomb_chance"]
    self.speed_scale = settings["speed_scale"]

def start_game(self):
    self.fruits = []
    self.trail = []

    self.score = 0
    self.lives = 3
    self.game_over_reason = None
    self._spawn_timer = 0

    self.apply_difficulty()
    self.state = "playing"

def end_game(self, reason):
    """Finish the game and play the Game Over sound once."""
```

```
    if self.state != "game_over":
        self.state = "game_over"
        self.game_over_reason = reason
        self.play_sound(self.game_over_sound)

def spawn_fruit(self):
    x = random.randint(60, self.width - 60)

    vy = -random.uniform(13, 16) * self.speed_scale
    vx = random.uniform(-2, 2)
    gravity = 0.35

    kind = (
        "bomb"
        if random.random() < self.bomb_chance
        else "fruit"
    )

    fruit = Fruit(
        x,
        self.height + 30,
        vx,
        vy,
        gravity,
        kind=kind,
    )

    fruit.color = (
```

```
        BOMB_BLACK
        if kind == "bomb"
        else random.choice(FRUIT_COLORS)
    )

    self.fruits.append(fruit)

def handle_event(self, event):

    if event.type == pygame.KEYDOWN:

        if self.state == "menu":

            if event.key == pygame.K_1:
                self.selected_difficulty = "Easy"

            elif event.key == pygame.K_2:
                self.selected_difficulty = "Medium"

            elif event.key == pygame.K_3:
                self.selected_difficulty = "Hard"

            elif event.key in (
                pygame.K_RETURN,
                pygame.K_SPACE,
            ):
                self.start_game()
```

```
        elif self.state == "game_over":

            if event.key in (
                pygame.K_r,
                pygame.K_RETURN,
            ):
                self.start_game()

            elif event.key == pygame.K_m:
                self.state = "menu"

    if self.state == "playing":

        if event.type == pygame.MOUSEMOTION:
            self._handle_motion(event.pos)

def _handle_motion(self, pos):
    previous_pos = self.trail[-1] if self.trail else pos

    for fruit in self.fruits:

        if fruit.sliced:
            continue

        intersects = segment_intersects_circle(
            previous_pos,
            pos,
            (fruit.x, fruit.y),
```

```
                fruit.radius,
            )

            if intersects:
                self._slice(fruit)

                if self.state != "playing":
                    break

        self.trail.append(pos)

        if len(self.trail) > 15:
            self.trail.pop(0)

    def _slice(self, fruit):

        if fruit.sliced:
            return

        fruit.sliced = True

        if fruit.kind == "bomb":
            self.play_sound(self.bomb_sound)
            self.end_game("You sliced a bomb!")

        else:
            self.score += 1
            self.play_sound(self.slice_sound)
```

```
def handle_input(self):
    pass

def update(self):

    if self.state != "playing":
        return

    self._spawn_timer += 1

    if self._spawn_timer >= self.spawn_interval:
        self._spawn_timer = 0
        self.spawn_fruit()

    still_alive = []

    for fruit in self.fruits:

        fruit.update()

        if fruit.sliced:
            continue

        if fruit.off_screen(self.height):

            if fruit.kind == "fruit":
                self.lives -= 1
```

```
            continue

        still_alive.append(fruit)

    self.fruits = still_alive

    if self.lives <= 0:
        self.lives = 0
        self.end_game("You ran out of lives!")

def render(self, screen):

    if self.state == "menu":
        self._render_menu(screen)
        return

    for fruit in self.fruits:

        color = getattr(fruit, "color", WHITE)

        pygame.draw.circle(
            screen,
            color,
            (int(fruit.x), int(fruit.y)),
            fruit.radius,
        )
```

```
if self.state == "playing" and len(self.trail) >= 2:

    pygame.draw.lines(
        screen,
        WHITE,
        False,
        self.trail,
        3,
    )

score_text = self.font.render(
    f"Score: {self.score}",
    True,
    WHITE,
)

screen.blit(score_text, (10, 10))

lives_text = self.font.render(
    f"Lives: {self.lives}",
    True,
    WHITE,
)

screen.blit(
    lives_text,
    (self.width - 130, 10),
)
```

```
    if self.state == "game_over":
        self._render_game_over(screen)

def _render_menu(self, screen):

    title = self.title_font.render(
        "FRUIT SLICE",
        True,
        (255, 200, 70),
    )

    screen.blit(
        title,
        title.get_rect(
            center=(self.width // 2, 140)
        ),
    )

    subtitle = self.message_font.render(
        "Choose your difficulty",
        True,
        WHITE,
    )

    screen.blit(
        subtitle,
        subtitle.get_rect(
```

```
            center=(self.width // 2, 220)
        ),
    )

    options = [
        ("1", "Easy", (100, 220, 120)),
        ("2", "Medium", (100, 180, 255)),
        ("3", "Hard", (255, 100, 100)),
    ]

    for index, (key, name, color) in enumerate(options):

        y = 290 + index * 55
        label = f"{key}. {name}"

        if name == self.selected_difficulty:
            label += "  (Selected)"

        text = self.font.render(
            label,
            True,
            color,
        )

        screen.blit(
            text,
            text.get_rect(
                center=(self.width // 2, y)
```

```
            ),
        )

    message = self.small_font.render(
        "Press ENTER or SPACE to start",
        True,
        WHITE,
    )

    screen.blit(
        message,
        message.get_rect(
            center=(self.width // 2, 490)
        ),
    )

def _render_game_over(self, screen):

    overlay = pygame.Surface(
        (self.width, self.height),
        pygame.SRCALPHA,
    )

    overlay.fill((0, 0, 0, 190))
    screen.blit(overlay, (0, 0))

    title = self.title_font.render(
        "GAME OVER",
```

```
    True,
    (255, 90, 90),
)

screen.blit(
    title,
    title.get_rect(
        center=(
            self.width // 2,
            self.height // 2 - 100,
        )
    ),
)

reason = self.message_font.render(
    self.game_over_reason or "The game has ended.",
    True,
    WHITE,
)

screen.blit(
    reason,
    reason.get_rect(
        center=(
            self.width // 2,
            self.height // 2 - 45,
        )
    ),
```

```
)

final_score = self.font.render(
    f"Final Score: {self.score}",
    True,
    WHITE,
)

screen.blit(
    final_score,
    final_score.get_rect(
        center=(
            self.width // 2,
            self.height // 2,
        )
    ),
)

replay = self.message_font.render(
    "Press R or ENTER to Play Again",
    True,
    (100, 255, 150),
)

screen.blit(
    replay,
    replay.get_rect(
        center=(
```

```
                self.width // 2,
                self.height // 2 + 55,
            )
        ),
    )

    menu = self.message_font.render(
        "Press M for Difficulty Menu",
        True,
        (180, 200, 255),
    )

    screen.blit(
        menu,
        menu.get_rect(
            center=(
                self.width // 2,
                self.height // 2 + 95,
            )
        ),
    )
```

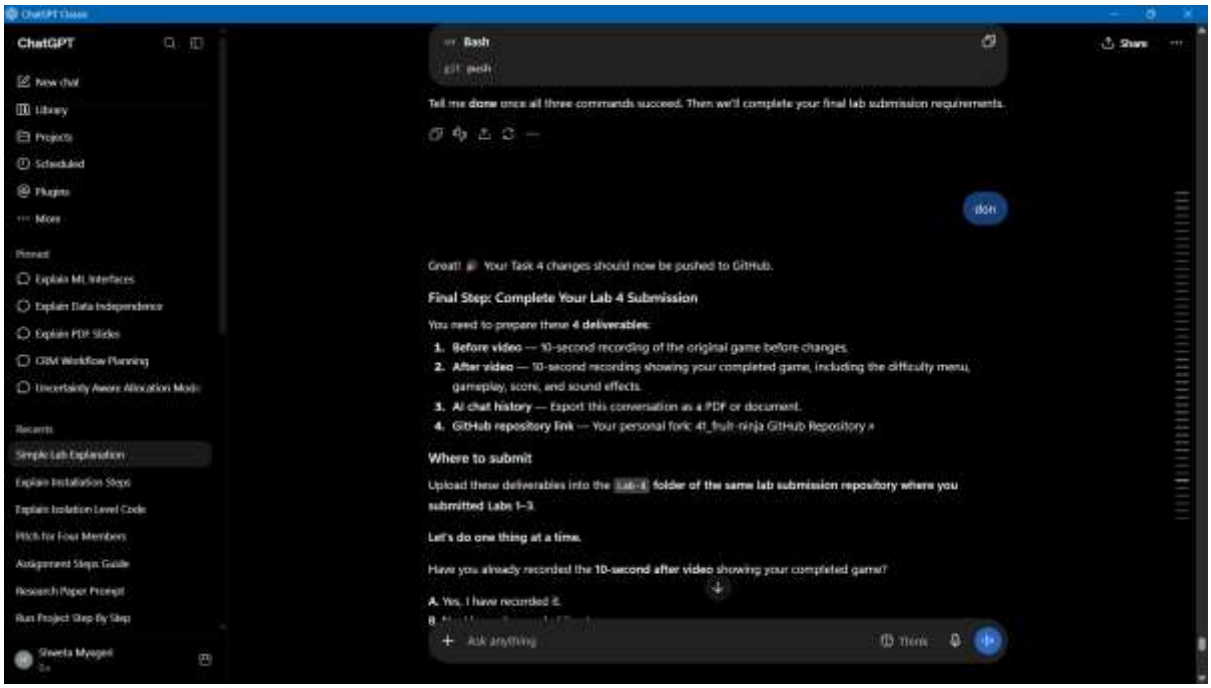
Step 2: Save and test

1. Press **Ctrl + S**.
2. Open the VS Code terminal.
3. Run:

```
python main.py
```

Check that the game starts and the difficulty menu appears. Start a game and slice a fruit to test the sound.

Tell me whether the game runs properly or displays an error. Don't commit yet—we'll test the sound effects first.



Excellent! 🎉 📢 **Task 4 is working, and all four tasks are now implemented.**

Next step: Commit Task 4 to GitHub

In your VS Code terminal, run these commands one at a time.

1. Stage the changes

git add game/game_engine.py

2. Commit Task 4

git commit -m "Task 4: Add sound effects"

3. Push to GitHub

git push

After all three commands succeed, tell me **done**. Then we'll finish the remaining lab submission requirements, including your after-video and chat history export.